

Part 1 of project 2 - answers

we are testing on the following sentences

1. "The Big Data platform for students is Blackboard"
2. "Questions on MinHash project by NTNU students is on Piazza"
3. "NTNU Big Data platform are Blackboard and Piazza"
4. "The project data for students are on Blackboard not Piazza"

1.1 Extracting K-Shingles

The 1-shingles from the first to the fourth sentence, disregarding all words that isnt adjectives or nouns, is as following:

```
[ "big", "data", "platform", "students", "blackboard" ] #1
[ "questions", "minhash", "project", "ntnu", "students", "piazza" ] #2
[ "ntnu", "big", "data", "platform", "blackboard", "piazza" ] #3
[ "project", "data", "students", "blackboard", "piazza" ] #4
```

1.2 Creating UniqueWords Dictionary

by combining the shingles, removing all duplicates and sorting alphabetically, we get the following list of unique words:

```
[ 'big', 'blackboard', 'data', 'minhash', 'ntnu', 'piazza', 'platform',
  'project', 'questions', 'students' ]
```

1.3 Input Matrix of Sentences

This is the input matrix generated from the k-shingles and the unique words dictionary. The first row is the first sentence

```
[
  [1, 1, 1, 0, 0, 0, 1, 0, 0, 1], #1 sentence
  [0, 0, 0, 1, 1, 1, 0, 1, 1, 1], #2 sentence
  [1, 1, 1, 0, 1, 1, 1, 0, 0, 0], #3 sentence
  [0, 1, 1, 0, 0, 1, 0, 1, 0, 1]  #4 sentence
]
```

Jaccard similarity for all combinations:

Pair	Jaccard Similarity
1 & 2	1/10
1 & 3	4/7
1 & 4	3/7
2 & 3	2/10
2 & 4	3/8
3 & 4	3/8

Sentence 1 and 3 has the most similarity

1.4 Calculating MinHash Signature matrix

We have three hash functions, which are lists of indices between 1 and 10. Hashing with these will create permutations by reordering the shingles.

Original Index	H1	H2	H3
1	5	9	10
2	6	6	7
3	7	3	4
4	8	10	1
5	9	7	8
6	10	4	5
7	1	1	2
8	2	8	9
9	3	5	6
10	4	2	3

In min hash we want to find the first index, where the value is 1 for each sentence and each hash-function. I will use 1 indexing for this case:

hashfunc	sentence 1	sentence 2	sentence 3	sentence 4
h1	1	2	1	2
h2	1	2	1	2
h3	2	1	2	3

Computing similarity from min hash-matrix

using the three hashes instead of all the shingles lets us do this much more compressed.

Pair	Jaccard-similarity
1&2	0
1&3	100%
1&4	0
2&3	0
2&4	66%
3&4	0

1&3 Is still the most similar sentences!